

Problem X Neural Networks

Consider the neural network with one hidden layer shown in Figure 1.

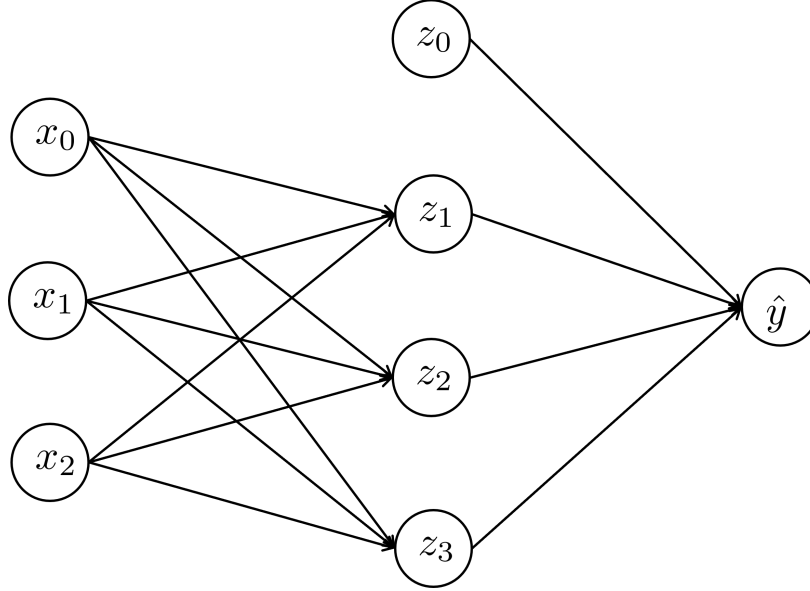


Figure 1: Neural Network

The input layer consists of 2 features $x = [x_1, x_2]^T$, the hidden layer has 3 nodes with output $z = [z_1, z_2, z_3]^T$, and the output layer is a scalar $\hat{y}$. There is a bias term for each node.

α is the matrix of weights from the inputs to the hidden layer. β is the matrix of weights from the hidden layer to the output layer. We will use a tanh activation function for the hidden layer and no activation for the output layer.

Forward Propagation is the process of calculating the value of your loss function, given data, weights and activation functions. Given the input data x , we can transform it by the given weights, α , then apply the corresponding activation function to it and finally pass the result to the next layer. Forward propagation does not involve taking derivatives and proceeds from the input layer to the output layer. In detail, we define each layer of the network as follows.

The input:

$$\mathbf{x} = [x_0, x_1, x_2]^T, \text{ where } x_0 = 1 \quad (1)$$

Linear combination at the first (hidden) layer for z_j node:

$$v_j = \sum_{i=0}^2 \alpha_{j,i} \cdot x_i \quad (2)$$

Activation at the first (hidden) layer:

$$z_j = \tanh(v_j) = \frac{e^{v_j} - e^{-v_j}}{e^{v_j} + e^{-v_j}} \quad (3)$$

$$\mathbf{z} = [z_0, z_1, z_2, z_3]^T, \text{ where } z_0 = 1 \quad (4)$$

Linear combination at the second (output) layer, we don't have activation for output layer:

$$\hat{y} = \sum_{j=0}^3 \beta_j \cdot z_j \quad (5)$$

We will use Squared error loss for each data point, $l(\hat{y}, y)$:

$$l(\hat{y}, y) = \frac{1}{2}(\hat{y} - y)^2 \quad (6)$$

We initialize the network weights α as:

$$\alpha = \begin{array}{ccc|c} & x_0 & x_1 & x_2 \\ \begin{array}{c} z_1 \\ z_2 \\ z_3 \end{array} & \begin{bmatrix} 0 & 1 & 2 \\ 2 & 1 & 0 \\ 0 & 2 & 0 \end{bmatrix} \end{array}$$

where each rows means the weights of a node z in hidden layer (e.g., the first row $[0, 1, 2]$ means the weights for x_0, x_1, x_2 in first hidden layer neuron z_1). The network weights β is:

$$\beta = \begin{array}{cccc|c} & z_0 & z_1 & z_2 & z_3 \\ & 0 & 1 & 2 & 2 \end{array} \hat{y}$$

where each number means the weights of a hidden layer neuron z (e.g., the second number 1 indicates the weight of z_1 in output layer).

1 [X points]. Why and how do we include a bias term in the input and in the hidden-layer?

[The bias terms helps the neural network better fit its data](#)

2 [X points]. Why do we need to use nonlinear activation functions in our neural net?

[A neural network with only linear activation functions would be no different than a linear regression](#)

3 [X points]. Given a data point $x_1 = 1, x_2 = 2, y = 3$, what are the values of v_1, v_2 , and v_3 ? What are the values of z_1, z_2 , and z_3

$$\begin{aligned} v_1 &= \sum_{i=0}^2 \alpha_{0,i} \cdot x_i = 5 \\ v_2 &= \sum_{i=0}^2 \alpha_{1,i} \cdot x_i = 3 \\ v_3 &= \sum_{i=0}^2 \alpha_{2,i} \cdot x_i = 2 \end{aligned}$$

by checking the look-up table:

$$z_1 = 0.99991$$

$$z_2 = 0.99505$$

$$z_3 = 0.96403$$

4 [X points]. What are the values of $\hat{y}$ and l ?

$$\hat{y} = 4.91807$$

$$l = 1.83950$$

Backpropagation Given a neural network and corresponding loss function for all data points $\mathcal{L}(\alpha, \beta)$, backpropagation gives us the gradient of the loss function with respect to the weights of the neural network. The method is called backward propagation because we calculate the gradients of the final layer of weights first, then proceed backward to the first layer. In this simple neural network with one hidden layer, the partial derivatives that we need for learning are $\frac{\partial \mathcal{L}}{\partial \alpha_{j,i}}$ and $\frac{\partial \mathcal{L}}{\partial \beta_k}$. For the weights before hidden layer like $\alpha_{j,i}$, we need to apply chain rule recursively for gradients.

5 [X points]. How many parameters will be updated in backpropagation?

9 weights and 4 bias

6 [X points]. Write the factorized form of $\frac{\partial \mathcal{L}}{\partial \alpha_{0,1}}$ with chain rule ($\alpha_{0,1}$ indicates the weights of x_1 at z_1)? Hint: $\frac{\partial \mathcal{L}}{\partial \alpha_{0,1}} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1} \frac{\partial z_1}{\partial v_1} \frac{\partial v_1}{\partial \alpha_{0,1}}$; $\frac{\partial \mathcal{L}}{\partial \beta_1} = \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \beta_1}$; $\frac{\partial \tanh(x)}{\partial x} = 1 - \tanh(x)^2$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \alpha_{0,1}} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial z_1} \frac{\partial z_1}{\partial v_1} \frac{\partial v_1}{\partial \alpha_{0,1}} \\ &= (\hat{y} - y) \beta_1 (1 - \tanh(v_1)^2) x_1 \end{aligned}$$

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \beta_1} &= \frac{\partial \mathcal{L}}{\partial \hat{y}} \frac{\partial \hat{y}}{\partial \beta_1} \\ &= (\hat{y} - y) z_1 \end{aligned}$$

7 [X points]. Given $x_1 = 1$, $x_2 = 2$, and $y = 3$, calculate the gradients of $\frac{\partial \mathcal{L}}{\partial \alpha_{0,1}}$ and $\frac{\partial \mathcal{L}}{\partial \beta_1}$.

$$\begin{aligned} \frac{\partial \mathcal{L}}{\partial \alpha_{0,1}} &= (\hat{y} - y) \beta_1 (1 - \tanh(v_1)^2) x_1 = (4.91807 - 3) \times 1 \times (1 - \tanh(5)^2) \times 1 \\ \frac{\partial \mathcal{L}}{\partial \beta_1} &= (\hat{y} - y) z_1 = (4.91807 - 3) \times 0.99991 = 1.9179 \end{aligned}$$

x	1	2	3	4	5	6	7	8	9
tanh(x)	0.76159	0.96403	0.99505	0.99933	0.99991	0.99999	0.99999	0.99999	0.99999

Figure 2: tanh values